

TorusNet 6.0 FR

L'Architech (Gabriel Breton-Breault)

Avant-propos

Ce document présente la structure directionnelle du protocole TorusNet telle qu'elle a été développée au fil des versions successives. Toutefois, il est important de noter qu'une **mise à jour majeure** de la structure interne des clés directionnelles a été introduite récemment. Cette mise à jour, identifiée comme **MAJ 6**, redéfinit entièrement la taille, l'organisation et le fonctionnement des clés, incluant l'introduction d'une clé de **128 bits** et d'un **moteur d'exposants multi-étages**.

Afin de préserver l'intégrité historique du document, les chapitres initiaux ont été conservés dans leur forme originale. Cependant, la **MAJ 6**, présentée à la fin du document, constitue désormais la **référence officielle** pour la structure des clés directionnelles et pour les mécanismes internes du seed directionnel.

En cas de divergence entre les chapitres et la mise à jour, c'est la MAJ 6 qui prévaut. Elle doit être considérée comme la version la plus récente, la plus complète et la plus exacte du protocole TorusNet.

Cette approche permet de maintenir la continuité du développement tout en offrant une vision claire et actualisée de l'architecture directionnelle utilisée par TorusNet.

Chapitre 1 — Introduction à l'Architecture Directionnelle

1.1. Vision générale

L'architecture directionnelle est une technologie de communication et de sécurité entièrement nouvelle. Elle ne repose pas sur des mathématiques, des dérivations cryptographiques ou des structures classiques, mais sur une **mécanique interne vivante**, impossible à dériver, impossible à explorer, impossible à falsifier.

Elle s'appuie sur trois éléments fondamentaux :

1. **Le Tore Privé** — une structure interne de 1 000 000 cases, chacune contenant :
 - **1 bit normal**,
 - **4 directions possibles**,

- le bit fantôme étant simplement l'inverse du bit normal. Le tore privé est unique à chaque session.
- 2. **La Mécanique Directionnelle** — un ensemble de règles qui définissent comment une clé se déplace dans le tore, comment les IDs mutent, comment les chemins internes s'activent, et comment la cohérence interne est maintenue.
- 3. **La Forteresse de Verre** — un module logiciel interne qui valide la cohérence directionnelle et bloque les IP hostiles au niveau du système d'exploitation.

L'architecture directionnelle n'utilise :

- ni chiffrement,
- ni dérivation mathématique,
- ni signatures,
- ni certificats internes,
- ni clés privées,
- ni primitives cryptographiques.

Elle repose exclusivement sur :

- des **fenêtres directionnelles**,
- des **constantes directionnelles**,
- des **parcours internes**,
- des **mutations d'ID**,
- des **chemins internes**,
- des **paires normal/fantôme**,
- des **structures internes non dérivables**.

1.2. Pourquoi une architecture directionnelle ?

Les protocoles classiques (TLS, SSH, IPSec, WireGuard, etc.) reposent tous sur :

- des mathématiques,
- des dérivations,
- des clés privées,

- des certificats,
- des signatures,
- des primitives cryptographiques.

Ces systèmes sont vulnérables à :

- l'analyse mathématique,
- les ordinateurs quantiques,
- les ordinateurs photoniques,
- les attaques par dérivation,
- les collisions,
- les attaques par répétition,
- les injections,
- les falsifications.

L'architecture directionnelle élimine entièrement ces vecteurs d'attaque.

Elle ne repose sur **aucune structure mathématique exploitable**. Elle repose sur une **mécanique interne**, impossible à dériver, impossible à explorer, impossible à inverser.

Même avec une puissance infinie, un attaquant ne peut pas :

- dériver un tore privé,
- explorer un tore privé,
- reconstruire un tore privé,
- prédire un parcours directionnel,
- falsifier une clé directionnelle,
- répéter une clé directionnelle,
- reconstruire un ID interne.

1.3. Les trois éléments fondamentaux

1.3.1. Le Tore Privé

Le Tore Privé est une structure interne de **1 000 000 cases**, chacune contenant :

- **1 bit normal,**
- **1 bit fantôme implicite** (NOT du bit normal),
- **4 directions possibles.**

Il est :

- unique à chaque session,
- reconstruit fractalement,
- jamais transmis,
- jamais exporté,
- jamais dérivable.

Il contient :

- les IDs internes,
- l'ID global,
- l'ID unique du tore,
- les chemins internes,
- les paires normal/fantôme.

1.3.2. La Mécanique Directionnelle

La mécanique directionnelle définit :

- comment une clé se déplace dans le tore,
- comment les IDs mutent,
- comment les chemins internes s'activent,
- comment les paires normal/fantôme s'inversent,
- comment la cohérence interne est maintenue.

Elle est :

- non linéaire,
- non dérivable,
- non réversible,

- vivante,
- oscillante,
- mutante.

1.3.3. La Forteresse de Verre

La Forteresse de Verre est un module logiciel interne qui :

- valide la cohérence directionnelle,
- surveille les ports directionnels,
- surveille les IDs internes,
- surveille l'ID global,
- surveille l'ID unique du tore,
- détruit la session en cas d'anomalie,
- bloque les IP hostiles dans le pare-feu du système d'exploitation.

Elle est l'équivalent directionnel d'un pare-feu structurel.

1.4. Ports directionnels : ports numériques étendus

Les ports directionnels sont :

- des **ports numériques classiques**,
- mais avec une plage **étendue dans les milliards**,
- générés par la clé directionnelle,
- validés par la Forteresse de Verre.

Ils servent à :

- transporter les transmissions directionnelles,
- éviter les collisions,
- empêcher les scans,
- empêcher les injections,
- empêcher les détournements de flux.

Un port directionnel peut ressembler à :

- 1 482 993 112
- 7 000 221 991
- 3 991 000 000

Impossible à scanner. Impossible à prédire. Impossible à falsifier.

1.5. Une architecture post-Internet

L'architecture directionnelle est conçue pour l'ère :

- post-quantique,
- post-cryptographique,
- photonique.

Elle ne dépend pas :

- des mathématiques,
- des certificats,
- des clés privées,
- des primitives cryptographiques.

Elle utilise encore :

- **DNS,**
- **HTTPS,**

mais uniquement comme **sas d'entrée temporaire**, avant que la session ne devienne 100 % directionnelle.

1.6. Une session directionnelle : vivante et autonome

Une session directionnelle est :

- vivante,
- autonome,
- auto-cohérente,
- auto-validante,
- auto-mutante.

Elle évolue à chaque clé :

- mutation d'ID,
- inversion normal/fantôme,
- progression interne,
- mise à jour du tore privé,
- validation par la Forteresse de Verre.

Elle ne peut pas être :

- copiée,
- clonée,
- dérivée,
- prédite,
- falsifiée.

1.7. Objectifs du protocole directionnel

L'architecture directionnelle vise à :

1. **Créer des sessions inviolables par conception** (aucune dérivation possible).
2. **Remplacer les protocoles cryptographiques classiques** (TLS, SSH, IPSec, WireGuard).
3. **Créer des réseaux privés directionnels** (multi-tore, branches indépendantes).
4. **Permettre la reconstruction directionnelle de fichiers** (bit par bit, fractalement).
5. **Créer une couche de communication interne** (ports directionnels numériques étendus).
6. **Créer une identité directionnelle unique** (ID interne, ID global, ID unique du tore).
7. **Assurer une sécurité post-quantique et photonique** (aucune structure mathématique exploitable).

1.8. Résumé du Chapitre 1

L'architecture directionnelle :

- n'est pas cryptographique,

- n'est pas mathématique,
- n'est pas dérivable,
- n'est pas falsifiable.

Elle repose sur :

- un tore privé,
- une mécanique interne,
- une cohérence directionnelle,
- des ports numériques étendus,
- une Forteresse de Verre logicielle.

Elle constitue une nouvelle génération de protocole, conçue pour l'ère post-quantique et photonique, où les mathématiques ne suffisent plus.

Chapitre 2 — Structure d'une Clé Directionnelle (64 bits)

2.1. Vue d'ensemble

La clé directionnelle est l'unité fondamentale du protocole directionnel. Chaque clé contient **64 bits**, organisés en segments stricts, chacun ayant un rôle précis dans :

- la sélection de la fenêtre directionnelle,
- la portée du parcours,
- la génération du port directionnel étendu,
- la mutation d'ID,
- la progression interne dans le Tore Privé,
- la cohérence directionnelle de la session.

Une clé directionnelle n'est **pas** une clé cryptographique. Elle ne sert **pas** à chiffrer, dériver ou signer. Elle sert à **déplacer, muter, valider, progresser**.

2.2. Décomposition des 64 bits

Une clé directionnelle est composée de **5 segments** :

Segment	Taille	Description
Fenêtre directionnelle (départ)	8 bits	Position N^1E^1
Fenêtre directionnelle (arrivée)	8 bits	Position N^2E^2
Constante directionnelle	2 bits	Portée du parcours
Seed directionnel	30 bits	Tour d'exposants (3×10 bits)
ID de la clé	16 bits	Identité unique de la clé

Total : **64 bits**

2.3. Fenêtre directionnelle (16 bits)

La fenêtre directionnelle est composée de :

- **8 bits de départ** : N^1E^1
- **8 bits d'arrivée** : N^2E^2

Elle détermine :

- la case de départ dans le Tore Privé,
- la case d'arrivée,
- la zone interne parcourue,
- la cohérence du déplacement,
- la validité de la clé.

La fenêtre directionnelle est la **géométrie brute** de la clé.

2.4. Constante directionnelle (2 bits)

Les 2 bits de constante directionnelle définissent la **portée du parcours**.

Bits	Constante	Portée
01	2	Courte
10	10	Moyenne

Bits Constante Portée

11 100 Large

00 1000 Maximale

La constante directionnelle :

- amplifie le seed,
- détermine la profondeur du parcours,
- influence la mutation d'ID,
- influence la zone du tore parcourue.

2.5. Seed directionnel (30 bits)

Les 30 bits du seed directionnel représentent une **tour d'exposants directionnelle**, composée de **trois exposants de 10 bits chacun** :

$$\text{Seed} = A^{B^C}$$

où :

- **A** = 10 bits
- **B** = 10 bits
- **C** = 10 bits

Total : **30 bits**

Ce seed n'est pas un nombre classique : → c'est une **structure exponentielle directionnelle**, utilisée pour amplifier la constante directionnelle et générer la base du parcours interne.

Rôle du seed directionnel

Le seed :

- est combiné avec la constante directionnelle,
- forme une tour d'exposants directionnelle,
- est converti en binaire,
- devient une séquence de déplacement interne,

- détermine les directions prises dans le tore privé,
- influence la mutation d'ID,
- influence la profondeur du parcours.

Le seed directionnel est la **source directionnelle brute** de la clé.

2.6. ID de la clé (16 bits)

Les 16 bits d'ID sont l'identité unique de la clé.

Ils servent à :

- valider la clé,
- empêcher les répétitions,
- empêcher les falsifications,
- empêcher les collisions,
- maintenir la cohérence de session,
- synchroniser les IDs internes,
- stabiliser l'ID global.

Une clé avec un ID incohérent détruit la session.

2.7. Mutation d'ID : les premiers bits du parcours

Règle fondamentale :

L'ID de mutation du Tore Privé (ID de la case de départ) est reconstruit à partir des premiers bits du parcours d'une clé dans le tore.

Cela signifie :

1. Dès que la clé commence son parcours dans le Tore Privé,
2. Les **tout premiers bits** utilisés pour déterminer normal/fantôme et direction,
3. Servent à **reconstruire l'ID de la case de départ**,
4. Avant même que le reste du parcours ne soit exécuté.

Pourquoi ?

Parce que :

- la mutation d'ID doit être immédiate,
- elle doit être cohérente avec la fenêtre directionnelle,
- cohérente avec la constante directionnelle,
- cohérente avec le seed,
- cohérente avec l'ID de la clé.

L'ID de la case de départ est donc :

- **le premier élément reconstruit,**
- **le premier élément muté,**
- **le premier élément validé,**
- **le premier élément surveillé par la Forteresse de Verre.**

Si cette mutation initiale est incohérente :

- la session est détruite,
- le tore privé est effacé,
- l'IP est bloquée.

2.8. Rôle global de la clé directionnelle

Une clé directionnelle :

- ouvre un port directionnel numérique étendu,
- détermine la fenêtre directionnelle,
- détermine la portée du parcours,
- génère le parcours interne,
- **mute immédiatement l'ID de la case de départ,**
- active les chemins internes,
- inverse normal/fantôme,
- met à jour le tore privé,
- valide la cohérence directionnelle.

Elle est le **moteur** de la session directionnelle.

2.9. Propriétés d'une clé directionnelle

Une clé directionnelle est :

- **non dérivable,**
- **non prédictible,**
- **non falsifiable,**
- **non répétable,**
- **non clonable,**
- **non inversable.**

Elle est :

- unique,
- autonome,
- vivante,
- mutante,
- auto-cohérente.

2.10. Résumé du Chapitre 2

La clé directionnelle :

- contient 64 bits,
- est composée de 5 segments,
- détermine la fenêtre directionnelle,
- détermine la portée du parcours,
- amplifie le seed,
- possède un ID unique,
- **reconstruit et mute l'ID de la case de départ dès les premiers bits du parcours,**
- génère le parcours interne,
- met à jour le tore privé,
- valide la cohérence directionnelle.

Elle est l'élément fondamental du protocole directionnel.

Chapitre 3 — Clé de Transmission (32 bits réseau + 80 bits internes + 240 bits directionnels)

3.1. Vue d'ensemble

La clé de transmission est la clé directionnelle **envoyée sur le réseau**. Elle fait **32 bits** et sert uniquement à :

- pointer la case de départ,
- pointer la case visée,
- fournir un seed directionnel réduit,
- permettre la reconstruction interne de **80 bits**,
- permettre ensuite la reconstruction d'un fichier de **n'importe quelle taille**.

✓ La clé réseau parcourt le tore

X Mais elle n'inverse aucun bit

X Elle ne mute aucune case

X Elle ne modifie rien

✓ Elle ne fait que lire

Seule la clé interne de 64 bits effectue les mutations.

3.2. Structure de la clé réseau (32 bits transmis)

La clé réseau contient **exactement 32 bits**, structurés ainsi :

3.2.1. 8 bits — Case de départ (4 bits entier + 4 bits exposant)

Ces 8 bits sont divisés en :

- **4 bits : entier**
- **4 bits : exposant**

Ils forment :

$$\text{CaseDépart} = \text{Entier}^{\text{Exposant}}$$

Ce couple sert à pointer la case de départ dans le tore privé.

3.2.2. 16 bits — ID de la case de départ (transmis)

Ces 16 bits représentent :

- l'ID de la case de départ (version réseau),
- l'identité initiale de la case avant mutation interne.

Cet ID est **transmis sur le réseau**.

3.2.3. 8 bits — Seed directionnel réduit (4 bits^4 bits)

Les derniers 8 bits sont découpés en :

- **4 bits : A**
- **4 bits : B**

Ils forment une mini tour d'exposants :

$$2^{A^B}$$

Ce résultat sert à reconstruire :

- les **16 bits internes** (nouvel ID de la case de départ),
- les **64 bits internes** (clé interne de reconstruction du fichier).

3.3. Reconstruction interne (80 bits)

À partir des 32 bits réseau, le tore privé reconstruit :

✓ **16 bits → nouvel ID interne de la case de départ**

✓ **64 bits → clé interne de reconstruction du fichier**

Total : **80 bits reconstruits**.

Ces 80 bits ne sont **jamais transmis**.

3.4. Mutation de la case de départ (16 bits)

Les **16 bits transmis** dans la clé réseau sont l'ID de la case de départ (version réseau).

Lorsque la clé réseau parcourt le tore :

Elle lit les 16 premiers bits reconstruits. Ces 16 bits reconstruits deviennent le nouvel ID interne de cette même case.

✓ **La clé réseau parcourt le tore**

X Mais elle n'inverse aucun bit

X Elle ne mute aucune case

X Elle ne modifie aucune structure interne

✓ Elle ne fait que lire

La clé interne mute la case.

3.5. Déplacements directionnels ($80 \times 3 = 240$ bits)

Les 80 bits reconstruits sont parcourus dans le tore privé via :

✓ 80 blocs de 3 bits

Chaque bloc contient :

Bit	Rôle
-----	------

1er bit	normal (0) ou fantôme (1)
---------	---------------------------

2e bit	direction interne — bit 1
--------	---------------------------

3e bit	direction interne — bit 2
--------	---------------------------

Les directions possibles :

- **00** → direction 1
- **01** → direction 2
- **10** → direction 3
- **11** → direction 4

Total :

$$80 \times 3 = 240 \text{ bits de déplacement}$$

✓ Seule la clé interne de 64 bits provoque ces déplacements

La clé réseau ne le fait jamais.

3.6. Reconstruction de la clé interne (64 bits)

Les 64 bits internes reconstruits :

- ne sont jamais transmis,

- ne quittent jamais le tore privé,
- servent à reconstruire le fichier,
- contiennent la géométrie directionnelle complète.

3.7. Reconstruction du fichier (taille illimitée)

Une fois la clé interne reconstruite :

- elle génère les déplacements nécessaires,
- elle reconstruit le fichier **bit par bit**,
- peu importe sa taille.

La taille du fichier :

- n'est jamais transmise,
- n'est jamais visible,
- est déduite directionnellement.

3.8. Validation par la Forteresse de Verre

La Forteresse de Verre valide :

- la clé réseau (32 bits),
- l'ID de la case de départ (16 bits transmis),
- le nouvel ID interne de la case (16 bits reconstruits),
- la reconstruction de la clé interne (64 bits),
- les 240 bits de déplacement,
- la cohérence du parcours,
- la reconstruction du fichier.

Si un seul élément est incohérent :

- session détruite,
- tore privé effacé,
- IP bloquée.

3.9. Résumé du Chapitre 3

La clé de transmission :

- transmet **32 bits** sur le réseau,
- parcourt le tore **sans muter**,
- reconstruit **80 bits internes** (16 + 64),
- génère **240 bits directionnels**,
- reconstruit un fichier de **n'importe quelle taille**,
- mute l'ID interne de la case de départ,
- valide la cohérence directionnelle,
- protège toutes les métadonnées sensibles.

Chapitre 4 — Forteresse de Verre (Pare-feu directionnel structurel)

4.1. Vue d'ensemble

La **Forteresse de Verre** est le pare-feu directionnel structurel du tore privé. Elle surveille et valide chaque opération directionnelle :

- la clé réseau (32 bits),
- les 16 bits transmis (ID de départ),
- les 16 bits reconstruits (nouvel ID interne),
- les 64 bits internes (clé interne),
- la cohérence fractale du tore,
- la reconstruction du fichier.

Elle est conçue pour rendre toute falsification **impossible par conception**.

4.2. Rôle fondamental

La Forteresse de Verre assure :

✓ Validation

Elle valide :

- la clé réseau,

- l’ID transmis,
- l’ID reconstruit,
- la clé interne,
- la cohérence fractale.

✓ Surveillance

Elle surveille :

- les inversions normal/fantôme (clé interne seulement),
- les mutations d’ID (clé interne seulement),
- les déplacements internes (clé interne seulement),
- les reconstructions directionnelles.

✓ Protection

Elle protège :

- le tore privé,
- la clé interne,
- la reconstruction du fichier,
- la cohérence directionnelle.

✓ Destruction

Si une incohérence est détectée :

- la session est détruite,
- le tore privé est effacé,
- l’IP est bloquée.

4.3. Validation de la clé réseau par le Host (point ajouté)

Avant que la Forteresse de Verre autorise **la moindre mutation interne**, elle vérifie :

si la clé réseau a été validée par le Host.

✓ Si le Host valide la clé réseau

La Forteresse de Verre autorise :

- la lecture des 80 bits reconstruits,
- la mutation des 16 bits internes,
- la reconstruction de la clé interne de 64 bits.

X Si le Host ne valide pas la clé réseau

La Forteresse de Verre :

- bloque toute mutation,
- bloque toute reconstruction,
- détruit la session.

C'est une règle directionnelle **absolue**.

4.4. Validation de la clé réseau (32 bits)

La Forteresse de Verre vérifie :

1. Les 8 bits de la case de départ

- entier^{exposant} valide
- case existante
- cohérence directionnelle

2. Les 16 bits de l'ID de départ

- ID existant
- ID non corrompu
- ID non falsifié

3. Les 8 bits du seed réduit

- A^B valide
- tour d'exposants cohérente
- absence d'anomalie directionnelle

Si un seul bit est incohérent : → session détruite.

4.5. Validation des 16 bits reconstruits (nouvel ID interne)

Les **16 premiers bits reconstruits** deviennent le **nouvel ID interne de la case de départ**.

La Forteresse de Verre vérifie :

- cohérence entre ID transmis et ID reconstruit,
- absence de mutation impossible,
- compatibilité avec la clé interne.

Si l'ID reconstruit est incohérent : → tore privé effacé.

4.6. Règle structurelle : les IDs de case peuvent se répéter

✓ Un ID de case peut se répéter

Jusqu'à **4 cases** peuvent partager le même ID.

✓ Cela ne compromet pas la sécurité

Car la sécurité repose sur :

- la géométrie directionnelle,
- les mutations internes,
- les déplacements directionnels,
- la cohérence fractale.

✓ Seulement 50% des 16 bits sont utilisés pour les IDs actifs

Les autres **50%** sont réservés aux **mutations directionnelles**.

✓ Les 1 000 000 de cases peuvent donc avoir des IDs répétés

Ce n'est pas une collision : → c'est une propriété directionnelle normale.

La Forteresse de Verre valide que :

- les répétitions sont légales,
- les mutations sont cohérentes.

4.7. Validation de la clé interne (64 bits)

La clé interne de 64 bits est **la seule clé qui mute et inverse**.

La Forteresse de Verre vérifie :

- cohérence interne,
- cohérence fractale,

- cohérence directionnelle,
- absence de falsification,
- absence de dérive.

Si la clé interne est incohérente : → session détruite.

4.8. Pas de validation des 240 bits de déplacement (point ajouté)

Tu l'as précisé :

La Forteresse de Verre ne valide pas les 240 bits de déplacement.

Donc :

✓ Les déplacements directionnels sont considérés comme cohérents

→ si la clé interne est valide

✓ La Forteresse ne vérifie pas chaque bloc de déplacement

→ elle vérifie seulement :

- la clé interne,
- l'ID reconstruit,
- la cohérence fractale globale.

✓ Les déplacements sont automatiques

→ ils ne nécessitent aucune validation individuelle.

4.9. Validation de la reconstruction du fichier

La Forteresse de Verre vérifie :

- cohérence de la clé interne,
- cohérence du tore,
- cohérence fractale,
- absence de falsification,
- absence de dérive.

Elle vérifie aussi que :

- la taille du fichier n'est jamais transmise,

- la taille du fichier n'est jamais visible,
- la reconstruction est purement directionnelle.

Si la reconstruction est incohérente : → session détruite.

4.10. Protection structurelle

La Forteresse de Verre protège :

✓ **Le tore privé**

✓ **La clé interne**

✓ **Le fichier**

✓ **La cohérence fractale**

Elle rend toute falsification **impossible par conception**.

4.11. Destruction automatique

Si une incohérence est détectée :

- la session est détruite,
- le tore privé est effacé,
- la clé interne est détruite,
- la reconstruction est annulée,
- l'IP est bloquée.

La destruction est :

- immédiate,
- irréversible,
- totale.

4.12. Résumé du Chapitre 4

La Forteresse de Verre :

- valide la clé réseau (32 bits),
- vérifie que le Host l'a validée,
- valide l'ID transmis (16 bits),

- valide l'ID reconstruit (16 bits),
- valide la clé interne (64 bits),
- n'a pas besoin de valider les 240 bits de déplacement,
- valide la reconstruction du fichier,
- accepte les IDs répétés,
- protège le tore privé,
- détruit la session en cas d'anomalie.

Elle est le **pare-feu directionnel structurel**, inviolable par conception.

Chapitre 5 — Tore Privé (Structure interne, géométrie, fichiers, directions, IDs)

5.1. Vue d'ensemble

Le **Tore Privé** est la structure directionnelle interne utilisée pour :

- reconstruire les 80 bits internes (16 + 64),
- parcourir les 240 bits directionnels,
- muter les IDs,
- inverser normal/fantôme,
- reconstruire le fichier,
- stabiliser la cohérence fractale.

Il contient :

- **1 000 000 cases**,
- chacune avec :
 - un **ID sur 16 bits**,
 - un **bit normal**,
 - un **bit fantôme implicite**,
 - **4 directions internes**,
 - un **déplacement torique**,

- un **saut exponentiel circulaire**.

Le tore privé est **programmé** avec deux fichiers distincts, lus en parallèle.

5.2. Structure interne du tore

Chaque case du tore contient :

✓ 1. Un ID sur 16 bits

- 100% des valeurs possibles (0 à 65535) sont utilisées
- les IDs peuvent se répéter librement
- aucune contrainte d'unicité
- aucune contrainte de collision
- aucune contrainte de disponibilité

✓ 2. Un bit normal

- 0 = normal
- 1 = fantôme (implicite, jamais transmis)

✓ 3. Quatre directions internes

Codées sur 2 bits :

Code Direction

00 direction 1

01 direction 2

10 direction 3

11 direction 4

✓ 4. Cohérence fractale

Chaque case possède une cohérence interne qui :

- se met à jour à chaque mutation,
- se met à jour à chaque inversion normal/fantôme,
- se met à jour à chaque déplacement directionnel.

5.3. Déplacements cardinaux et diagonaux (comportement torique)

Les déplacements internes se comportent comme un **tore parfait** :

✓ Dépasser en haut → tu arrives en bas

✓ Dépasser en bas → tu arrives en haut

✓ Dépasser à gauche → tu arrives à droite

✓ Dépasser à droite → tu arrives à gauche

Ce comportement :

- supprime les bords,
- supprime les limites,
- supprime les zones mortes,
- garantit une continuité parfaite,
- rend impossible toute sortie du tore.

✓ Les diagonales suivent la même logique

Un déplacement diagonal combine deux déplacements toriques simultanés.

5.4. Sauts exponentiels (comportement circulaire)

Chaque direction contient aussi un **saut exponentiel**.

Ce saut :

- ne suit pas les axes cardinaux,
- ne suit pas les diagonales,
- ne suit pas la grille,
- ne suit pas la géométrie du tore.

Il suit une **orbite circulaire interne**.

✓ Le saut exponentiel se déplace sur un cercle

✓ Quand il arrive à la fin du cercle → il revient au début

✓ Il ne peut jamais sortir de l'orbite

✓ Il ne peut jamais se bloquer

✓ Il ne peut jamais se dériver

Ce comportement circulaire :

- amplifie la portée directionnelle,
- crée une géométrie interne indépendante,
- rend les parcours impossibles à inverser,
- stabilise la cohérence fractale,
- empêche toute dérivation externe.

5.5. Géométrie hybride : tore + cercle

Tu obtiens une structure **hybride** :

✓ **Géométrie torique**

Pour les déplacements cardinaux et diagonaux.

✓ **Géométrie circulaire**

Pour les sauts exponentiels.

Cette dualité :

- crée une surface directionnelle vivante,
- change de géométrie selon le type de déplacement,
- rend les parcours impossibles à inverser,
- stabilise la cohérence fractale,
- empêche toute dérivation externe.

5.6. Architecture logicielle du Tore Privé

Le tore privé est **programmé** avec deux fichiers distincts, chacun ayant un rôle précis :

5.6.1. Fichier des bits normaux (binaire pur, chargé en RAM)

Ce fichier :

- contient **uniquement les bits normaux**,
- est un **fichier binaire pur**,

- est **chargé entièrement en RAM** lors de l'utilisation du tore,
- ne contient **aucune direction**,
- ne contient **aucun ID**,
- ne contient **aucun saut exponentiel**.

✓ Pourquoi en RAM ?

Parce que :

- la mutation des cases doit être **instantanée**,
- la lecture des bits normaux doit être **ultra-rapide**,
- la reconstruction du fichier doit être **continue**,
- les inversions normal/fantôme doivent être **immédiates**.

Séparer les bits normaux du reste :

rend la mutation des cases plus facile, plus rapide, plus stable.

5.6.2. Fichier directionnel (texte, reste sur le disque dur)

Ce fichier :

- reste **sur le disque dur**,
- est un **fichier texte**,
- contient **les 4 directions de chaque case**,
- contient **les sauts exponentiels**,
- contient **les IDs de chaque case**,
- contient **1 000 000 lignes**,
- est parcouru **en parallèle** avec le fichier binaire.

✓ Structure exacte d'une ligne du fichier directionnel

Tu l'as définie ainsi :

Code

<ID> 101 2^5, 110 3^7, 101 2^7, 111 4^7

Chaque ligne représente **une case** du tore privé.

Elle contient :

1. **L'ID de la case** (16 bits)
2. **Direction 1 :**
 - déplacement (3 bits)
 - saut exponentiel ($\text{entier}^{\text{exposant}}$)
3. **Direction 2 :**
 - déplacement (3 bits)
 - saut exponentiel
4. **Direction 3 :**
 - déplacement (3 bits)
 - saut exponentiel
5. **Direction 4 :**
 - déplacement (3 bits)
 - saut exponentiel

✓ Lecture parallèle

Pendant l'utilisation du tore :

- le fichier binaire (bits normaux) est lu en RAM,
- le fichier directionnel (déplacements + sauts + IDs) est lu sur le disque,
- les deux sont synchronisés en temps réel.

5.7. Mutation des IDs

Il existe **deux types de mutation** :

5.7.1. Mutation locale (clé réseau)

La clé réseau (32 bits) :

- parcourt le tore,
- lit les cases,
- reconstruit les 80 bits internes,

- **mute uniquement l'ID de la case où elle pointe,**
- ne touche pas l'ID du tore,
- n'inverse aucun bit normal/fantôme,
- ne mute aucune autre case.

Mutation locale :

$$ID_{case} \leftarrow ID_{reseau}$$

5.7.2. Mutation structurelle (clé interne)

La clé interne (64 bits) :

- parcourt le tore,
- inverse normal/fantôme,
- reconstruit le fichier,
- **mute l'ID du tore,**
- **mute l'ID de la case de départ,**
- modifie la cohérence fractale.

Mutation structurelle :

$$ID_{tore} \leftarrow ID_{aleatoire}$$

$$ID_{case\ départ} \leftarrow ID_{aleatoire}$$

Les IDs aléatoires sont choisis sur **16 bits**, même s'ils sont déjà utilisés.

5.8. Déplacements directionnels (240 bits)

Les 80 bits reconstruits (16 + 64) sont parcourus via :

✓ **80 blocs de 3 bits**

Chaque bloc contient :

- 1 bit normal/fantôme
- 2 bits direction

✓ **Total : 240 bits de déplacement**

Ces déplacements :

- mutent les IDs (clé interne seulement),
- inversent normal/fantôme (clé interne seulement),
- reconstruisent le fichier,
- mettent à jour la cohérence fractale.

✓ **La clé réseau parcourt le tore mais n'inverse rien**

Elle ne fait que lire.

5.9. Normal / Fantôme

Chaque case possède :

- un **bit normal**,
- un **bit fantôme implicite**.

✓ **La clé interne peut inverser normal/fantôme**

X **La clé réseau ne peut pas inverser normal/fantôme**

L'inversion :

- modifie la géométrie directionnelle,
- modifie la cohérence fractale,
- modifie les chemins internes.

5.10. ID du tore

Le tore possède un **ID global sur 16 bits**.

Cet ID :

- change à chaque clé interne,
- est choisi aléatoirement sur 16 bits,
- peut se répéter,
- n'a aucune contrainte d'unicité.

La clé réseau ne touche jamais l'ID du tore.

5.11. Rôle du tore dans la reconstruction du fichier

Le tore :

- reconstruit les 80 bits internes,
- génère les 240 bits directionnels,
- reconstruit la clé interne,
- reconstruit le fichier bit par bit,
- stabilise la cohérence fractale.

La taille du fichier :

- n'est jamais transmise,
- n'est jamais visible,
- est déduite directionnellement.

5.12. Résumé du Chapitre 5

Le Tore Privé :

- contient 1 000 000 cases,
- chaque case a un ID sur 16 bits,
- 100% des IDs sont utilisés,
- les IDs peuvent se répéter librement,
- les déplacements cardinaux sont toriques,
- les sauts exponentiels sont circulaires,
- le tore est programmé avec deux fichiers (binaire RAM + directionnel disque),
- la clé réseau parcourt sans muter,
- la clé interne parcourt en mutant,
- la géométrie hybride rend le tore inviolable.

Chapitre 6 — Système des ID (16 bits, répétition, mutation, rôle)

(Version V5.2 — complet)

6.1. Vue d'ensemble

Le protocole directionnel utilise un système d'identifiants (ID) entièrement basé sur :

- des IDs sur 16 bits,
- 100% des valeurs possibles,
- des IDs répétables,
- des mutations aléatoires,
- des redéfinitions locales et globales,
- des interactions distinctes entre clé réseau et clé interne.

Les IDs sont au cœur :

- du tore privé,
- des cases,
- des mutations,
- de la cohérence fractale,
- de la reconstruction du fichier.

6.2. Taille des IDs : 16 bits

Tous les IDs du protocole sont sur **16 bits** :

- ID de case
- ID du tore
- ID transmis par la clé réseau
- ID reconstruit par la clé interne
- ID muté par la clé interne
- ID muté par la clé réseau

✓ Valeurs possibles :

0 → 65535

✓ 100% des valeurs sont utilisées

Aucune valeur n'est réservée. Aucune valeur n'est interdite. Aucune valeur n'est mise de côté.

6.3. Répétition des IDs

Tu as établi une règle directionnelle majeure :

**Les IDs peuvent se répéter librement. Plusieurs cases peuvent partager le même ID.
Cela ne compromet rien pour la sécurité.**

✓ Pourquoi ?

Parce que :

- l'ID n'est **pas** un identifiant unique,
- l'ID est un **marqueur directionnel**,
- la sécurité repose sur la **géométrie directionnelle**,
- pas sur l'unicité des IDs.

✓ Conséquence

- 1 000 000 cases
- 65 536 IDs possibles
- Répétition normale
- Collisions normales
- Aucun impact sur la cohérence fractale

6.4. Mutation des IDs

Il existe **deux types de mutation** :

6.4.1. Mutation locale (clé réseau)

La clé réseau (32 bits) :

- parcourt le tore,
- lit les cases,
- reconstruit les 80 bits internes,
- **mute uniquement l'ID de la case où elle pointe,**
- ne touche pas l'ID du tore,
- ne mute aucune autre case.

Mutation locale :

$$ID_{\text{case pointee}} \leftarrow ID_{\text{reseau}}$$

6.4.2. Mutation structurelle (clé interne)

La clé interne (64 bits) :

- parcourt le tore,
- inverse normal/fantôme,
- reconstruit le fichier,
- **mute l'ID du tore,**
- **mute l'ID de la case de départ,**
- modifie la cohérence fractale.

Mutation structurelle :

$$ID_{\text{tore}} \leftarrow ID_{\text{aleatoire}}$$

$$ID_{\text{case depart}} \leftarrow ID_{\text{aleatoire}}$$

6.5. Mutation aléatoire sur 16 bits

Tu as établi une règle capitale :

Lors d'une mutation, le programme choisit un ID au hasard sur 16 bits, même si cet ID est déjà utilisé.

Donc :

- mutation = tirage aléatoire sur 0–65535
- aucune contrainte d'unicité
- aucune contrainte de collision
- aucune contrainte de disponibilité
- mutation purement directionnelle

6.6. Rôle des IDs dans le tore privé

Chaque case du tore possède :

- un ID sur 16 bits,
- un bit normal,

- un bit fantôme implicite,
- quatre directions internes,
- un déplacement torique,
- un saut exponentiel circulaire.

L'ID :

- identifie la case dans le fichier directionnel,
- sert de point d'entrée pour les mutations,
- sert de marqueur pour la cohérence fractale,
- sert de repère pour la reconstruction interne.

6.7. ID du tore

Le tore possède un **ID global sur 16 bits**.

Cet ID :

- change à chaque clé interne,
- est choisi aléatoirement,
- peut se répéter,
- n'a aucune contrainte d'unicité.

La clé réseau ne touche jamais l'ID du tore.

6.8. Interaction entre IDs et clés

✓ Clé réseau (32 bits)

- parcourt le tore,
- lit les cases,
- reconstruit les 80 bits internes,
- **mute uniquement l'ID de la case pointée,**
- ne touche pas l'ID du tore,
- ne mute aucune autre case.

✓ Clé interne (64 bits)

- parcourt le tore,
- inverse normal/fantôme,
- reconstruit le fichier,
- **mute l’ID du tore,**
- **mute l’ID de la case de départ,**
- stabilise la cohérence fractale.

6.9. Cohérence fractale et IDs

La cohérence fractale :

- accepte les IDs répétés,
- accepte les collisions,
- accepte les mutations aléatoires,
- se stabilise automatiquement,
- se met à jour à chaque clé interne.

Les IDs ne sont pas une structure de sécurité. Ils sont une structure **directionnelle**.

6.10. Résumé du Chapitre 6

Le système des IDs fonctionne ainsi :

- tous les IDs sont sur 16 bits,
- 100% des valeurs sont utilisées,
- les IDs peuvent se répéter librement,
- les mutations choisissent un ID aléatoire,
- la clé réseau mute seulement la case pointée,
- la clé interne mute le tore et la case de départ,
- les collisions d’ID sont normales,
- la cohérence fractale reste stable,
- la sécurité repose sur la géométrie directionnelle,
- pas sur l’unicité des IDs.

Chapitre 7 — Handshake Directionnel (Séquence Host → Client → Host → Client → Host)

7.1. Vue d'ensemble

Le handshake directionnel est le mécanisme fondamental qui établit une session cohérente entre un **Host** et un **Client**.

Contrairement aux protocoles classiques (TLS, SSH, IPSec) qui reposent sur :

- clés privées,
- certificats,
- signatures,
- dérivations mathématiques,
- échanges cryptographiques,

le handshake directionnel repose exclusivement sur :

- la géométrie interne des tores,
- la cohérence fractale,
- la cascade orbitale,
- les déplacements directionnels,
- les IDs internes,
- la validation structurelle.

Il ne contient **aucune cryptographie**, aucune dérivation, aucune structure mathématique exploitable.

7.2. Objectifs du handshake

Le handshake directionnel assure :

1. la validation de la première clé directionnelle,
2. la synchronisation initiale entre Host et Client,
3. l'inscription de l'IP du Client comme IP de confiance,
4. la création de l'ID global de session,

5. l'activation de la Forteresse de Verre,
6. la reconstruction initiale du Tore Privé,
7. la mise en place de l'ID unique du tore (16 bits),
8. la synchronisation directionnelle complète via **5 transmissions**.

Ces étapes sont **structurelles**, non cryptographiques.

7.3. La séquence officielle : 5 transmissions

La séquence directionnelle comporte **5 transmissions**, chacune ayant un rôle structurel précis. Elle est **obligatoire, inviolable**, et constitue la base de la synchronisation directionnelle.

Transmission 1 — Host → Client

Envoi initial de la clé directionnelle

Le Host envoie une clé de **64 bits** découpée en :

- **8 bits** : point de départ (N^E),
- **8 bits** : point d'arrivée (N^E),
- **2 bits** : type de clé,
- **30 bits** : anneau orbital,
- **16 bits** : ID de la case visée.

Le Client :

- reçoit la clé,
- vérifie sa structure,
- vérifie son type,
- vérifie l'ID visé,
- vérifie la cohérence fractale initiale.

Aucune lecture orbitale n'est encore exécutée.

Transmission 2 — Client → Host

Renvoi de la clé pour confirmer la réception

Le Client renvoie **exactement la même clé** au Host.

Le Host :

- compare la clé envoyée et la clé reçue,
- valide la réception,
- inscrit l'IP du Client comme **IP de confiance**,
- active la Forteresse de Verre.

Si la clé diffère → **destruction immédiate**.

Transmission 3 — Host → Client

Renvoi final de la clé pour autoriser l'exécution

Le Host renvoie une seconde copie de la clé.

Le Client :

- compare la clé renvoyée avec celle reçue,
- valide la cohérence,
- **exécute le parcours directionnel dans son tore privé**,
- **inverse les bits normal/fantôme**,
- **mute l'ID de la case de départ**,
- met à jour les chemins internes,
- complète la **cascade orbitale** (paliers 1 à 10).

Le Tore Privé du Client devient :

- vivant,
- oscillant,
- modifié.

Transmission 4 — Client → Host

Confirmation d'exécution du parcours

Le Client renvoie la clé après avoir exécuté le parcours.

Ce renvoi indique au Host :

- que les inversions normal/fantôme ont été faites,

- que l’ID de la case de départ a muté,
- que les chemins internes ont été mis à jour,
- que la progression fractale est cohérente,
- que le tore privé du Client est à jour.

Le Host peut alors :

- exécuter le même parcours,
- inverser normal/fantôme de son côté,
- muter l’ID de la case de départ,
- synchroniser son tore privé avec celui du Client.

Transmission 5 — Host → Client

Confirmation finale de synchronisation

Le Host renvoie la clé une dernière fois pour confirmer :

- qu’il a bien exécuté le parcours,
- qu’il a inversé normal/fantôme,
- qu’il a muté l’ID de la case de départ,
- que son tore privé est synchronisé avec celui du Client,
- que la session est directionnellement stable.

Le Client reçoit cette confirmation et la session devient :

- cohérente,
- stable,
- inviolable.

7.4. Pourquoi cinq transmissions ?

7.4.1. Validation bidirectionnelle complète

Host valide Client. Client valide Host. Puis chacun valide l’exécution directionnelle de l’autre.

7.4.2. Protection contre les injections

Un attaquant ne peut pas :

- intercepter la clé,
- la modifier,
- la renvoyer,
- la falsifier.

Toute incohérence est détectée **structurellement**.

7.4.3. Protection contre les répétitions

Une clé répétée :

- est détectée,
- est rejetée,
- détruit la session.

7.4.4. Protection contre les dérivations

Un attaquant ne peut pas dériver :

- le point de départ,
- le point d'arrivée,
- les 30 bits orbitaux,
- les déplacements exponentiels,
- l'ID de la case visée,
- la mutation d'ID.

La structure directionnelle est **non dérivable**.

7.4.5. Synchronisation directionnelle parfaite

Les 5 transmissions garantissent que :

- les deux tores privés ont exécuté le même parcours,
- les inversions normal/fantôme sont symétriques,
- les mutations d'ID sont identiques,
- la géométrie interne est cohérente,

- aucune divergence n'est possible.

7.5. Synchronisation initiale

Une fois la clé validée :

- le Tore d'Authentification est utilisé une seule fois,
- le Tore Privé est initialisé,
- l'ID global est créé,
- les paires normal/fantôme sont définies,
- les chemins internes sont activés,
- l'ID unique du tore est établi,
- la Forteresse de Verre est armée.

Cette synchronisation est **fractale**, non mathématique.

7.6. Reconstruction initiale du Tore Privé

La première clé déclenche :

- la génération de l'anneau orbital,
- les 60 lectures orbitales,
- les 180 blocs directionnels (N,E),
- la lecture des blocs de 3 bits dans le Tore Public,
- la mutation de l'ID de la case de départ,
- la mise à jour des chemins internes,
- la mise en place de la cohérence directionnelle.

Le Tore Privé devient :

- vivant,
- oscillant,
- opérationnel.

7.7. Gestion des erreurs pendant le handshake

7.7.1. Clé incohérente

Si la clé :

- contient un ID invalide,
- contient un point de départ impossible,
- contient un point d'arrivée impossible,
- contient un type illégitime,

→ la session est détruite.

7.7.2. Clé modifiée

Si la clé renvoyée par le Client diffère :

- destruction immédiate,
- effacement du Tore Privé,
- invalidation de l'IP.

7.7.3. Absence de réponse

Si le Host ne reçoit aucune réponse après 5 secondes :

- la clé est considérée perdue,
- une nouvelle clé est générée,
- avec un point de départ différent,
- mais reconstruisant **le même fichier directionnel**.

Aucune retransmission du même paquet n'est effectuée.

7.8. Résumé du Chapitre 7

Le handshake directionnel :

- repose sur **5 transmissions**,
- valide la première clé directionnelle,
- inscrit l'IP du Client comme IP de confiance,
- initialise le Tore Privé,
- active la Forteresse de Verre,
- crée l'ID global,

- établit l’ID unique du tore,
- garantit une cohérence directionnelle totale,
- protège contre les injections, répétitions, dérivations et falsifications.

Il constitue la base de la **sécurité structurelle directionnelle**.

Chapitre 8 — Multi-Tore et Réseaux Parallèles

8.1. Vue d’ensemble

Le multi-tore est l’un des fondements les plus puissants de l’architecture directionnelle. Il permet de créer plusieurs réseaux directionnels indépendants, chacun possédant :

- son propre Tore Privé,
- son propre Tore d’Authentification,
- ses propres clés directionnelles,
- ses propres IDs internes,
- son propre ID global,
- son propre ID unique du tore,
- sa propre Forteresse de Verre.

Le Tore Public, lui, est pré-généré. Il vient avec le logiciel du protocole et constitue la base directionnelle universelle pour toutes les branches.

Chaque réseau parallèle est totalement isolé des autres, même s’ils coexistent :

- sur la même machine,
- sur le même système,
- dans le même environnement réseau.

Le multi-tore permet :

- la segmentation directionnelle,
- la souveraineté structurelle,
- l’isolation totale des branches,
- la création de réseaux privés inviolables,

- la duplication directionnelle contrôlée.

8.2. Définition d'un réseau parallèle

Un réseau parallèle est une instance complète de l'architecture directionnelle comprenant :

1. Le Tore Public (pré-généré, universel)
2. Un Tore d'Authentification
3. Un Tore Privé
4. Un ensemble de clés directionnelles
5. Une Forteresse de Verre
6. Un ID global de session
7. Un ID unique du tore

Chaque réseau parallèle est un univers directionnel indépendant.

Il ne partage :

- aucune clé,
- aucun ID,
- aucun tore privé,
- aucune progression fractale,
- aucune cohérence interne.

8.3. Création d'un nouveau tore

La création d'une nouvelle branche directionnelle suit trois étapes :

8.3.1. Utilisation du Tore Public pré-généré

Le Tore Public n'est pas généré lors de la création d'une branche. Il est :

- pré-généré,
- fourni avec le logiciel,
- universel,
- identique pour toutes les installations,

- non modifiable,
- non dérivable.

Il contient toutes les combinaisons possibles des 30 bits, ce qui en fait la base directionnelle universelle.

8.3.2. Génération du Tore d'Authentification

Un Tore d'Authentification de 1 000 000 cases $\times$ 3 bits est généré. Il valide la première clé directionnelle et initialise :

- les premières paires normal/fantôme,
- les premiers chemins internes,
- l'ID global,
- l'ID unique du tore.

8.3.3. Génération du Tore Privé

Un Tore Privé de 1 000 000 cases $\times$ 3 bits est généré. Il sera reconstruit fractalement à partir des clés directionnelles :

- inversion normal/fantôme,
- mutation d'ID,
- activation des chemins internes,
- progression orbitale.

Chaque Tore Privé est indépendant des autres tores privés existants.

8.4. Branches directionnelles

Chaque réseau parallèle est appelé **branche directionnelle**.

Une branche possède :

- son propre Tore Privé,
- sa propre progression fractale,
- ses propres IDs internes,
- sa propre Forteresse de Verre,
- son propre ID global,

- son propre ID unique du tore.

8.4.1. Branches indépendantes

Les branches ne communiquent pas entre elles. Elles ne peuvent pas :

- partager des clés,
- partager des IDs,
- partager des tores privés,
- partager des chemins internes,
- partager des mutations d'ID.

8.4.2. Branches souveraines

Chaque branche est souveraine :

- elle ne dépend d'aucune autre branche,
- elle ne peut pas être influencée par une autre branche,
- elle ne peut pas être dérivée par une autre branche.

8.5. Duplication directionnelle (Tore primaire hors-ligne)

Une fonctionnalité essentielle du multi-tore est la **duplication directionnelle hors-ligne**.

8.5.1. Tore primaire

Un tore primaire peut être :

- généré,
- stocké hors-ligne,
- transféré physiquement,
- utilisé pour créer une nouvelle branche.

8.5.2. Transfert hors-ligne

Le tore primaire peut être transféré :

- sur clé USB,
- sur disque dur,
- sur support physique,

- sur tout média non réseau.

Ce transfert ne contient **aucune clé directionnelle**. Il contient uniquement :

- la géométrie directionnelle,
- les blocs de 3 bits,
- les chemins internes.

8.5.3. Création d'une branche à partir du tore primaire

Une fois le tore primaire transféré :

- une nouvelle branche est créée,
- un nouveau Tore Privé est reconstruit,
- un nouvel ID global est généré,
- un nouvel ID unique du tore est établi,
- une nouvelle Forteresse de Verre est activée.

La branche est totalement indépendante de la branche d'origine.

8.6. Isolation structurelle

L'isolation structurelle est totale.

8.6.1. Isolation fractale

Les actions fractales d'une branche :

- ne peuvent pas influencer une autre branche,
- ne peuvent pas dériver une autre branche,
- ne peuvent pas reconstruire une autre branche.

8.6.2. Isolation directionnelle

Les chemins internes d'une branche :

- ne peuvent pas être utilisés dans une autre branche,
- ne peuvent pas être dérivés par une autre branche.

8.6.3. Isolation des IDs

Les IDs internes :

- ne sont pas compatibles entre branches,
- ne peuvent pas être transférés,
- ne peuvent pas être dérivés.

8.7. Multi-tore et sécurité post-quantique

Le multi-tore renforce la sécurité post-quantique :

- un ordinateur quantique ne peut pas dériver un tore privé,
- ne peut pas dériver une branche,
- ne peut pas dériver une clé,
- ne peut pas dériver un ID,
- ne peut pas dériver l'ID unique du tore.

Même avec une puissance infinie, un attaquant ne peut pas :

- explorer un tore privé,
- dériver un tore privé,
- reconstruire un tore privé,
- analyser un tore privé.

Les tores privés sont **géométriquement inviolables**.

8.8. Multi-tore et sécurité photonique

Avec l'arrivée des ordinateurs photoniques :

- les structures mathématiques deviennent obsolètes,
- les cryptographies classiques deviennent vulnérables.

Le multi-tore est conçu pour cette ère :

- les tores privés sont non transmissibles,
- non dérivables,
- non analysables,
- non explorables.

Un ordinateur photonique ne peut pas :

- pénétrer un tore privé,
- analyser un tore privé,
- dériver un tore privé.

8.9. Résumé du Chapitre 8

Le multi-tore permet :

- la création de réseaux parallèles,
- l'isolation totale des branches,
- la duplication directionnelle hors-ligne,
- la souveraineté structurelle,
- la sécurité post-quantique,
- la sécurité photonique.

Chaque branche est un univers directionnel indépendant, inviolable par conception. Le Tore Public, pré-généré et universel, sert de base directionnelle commune, mais **jamais** de tore privé partagé.

Chapitre 9 — Ports Directionnels et Géométrie Interne de Communication

9.1. Vue d'ensemble

Les ports directionnels de TorusNet sont une extension radicale des ports classiques. Contrairement aux ports TCP/UDP (0–65535), TorusNet utilise :

- **des ports classiques étendus,**
- **plusieurs milliards de possibilités,**
- **mutables,**
- **non scannables,**
- **non dérivables,**
- **non falsifiables,**
- **validés structurellement par la Forteresse de Verre.**

Un port directionnel n'est pas :

- un simple entier,
- un socket,
- un canal TCP,
- un identifiant réseau.

C'est un **port étendu**, généré par le tore privé, qui change à chaque renouvellement du tore et à chaque nouvelle connexion.

9.2. Définition d'un port directionnel

Un port directionnel est :

- un **numéro de port étendu** (plusieurs milliards de possibilités),
- généré par le tore privé,
- muté à chaque renouvellement du tore,
- muté à chaque nouvelle connexion,
- impossible à prédire,
- impossible à scanner,
- impossible à dériver.

Règle structurelle : une seule connexion par port

Chaque port directionnel :

- n'autorise **qu'une seule connexion**,
- est invalidé dès que la session se termine,
- est remplacé par un nouveau port étendu lors de la prochaine connexion.

9.3. Pourquoi les ports directionnels sont inviolables

9.3.1. Ils n'existent pas dans la couche réseau classique

Un attaquant ne peut pas :

- scanner les ports,
- sonder les ports,
- ouvrir un port,

- envoyer un paquet vers un port.

Les ports directionnels n'existent **que dans la mécanique interne du tore**.

9.3.2. Ils sont mutables et non dérivables

Les ports directionnels :

- mutent automatiquement,
- ne sont jamais réutilisés,
- ne peuvent pas être prédits,
- ne peuvent pas être dérivés mathématiquement.

9.3.3. Ils sont validés par la Forteresse de Verre

La Forteresse de Verre :

- surveille les ports directionnels,
- valide leur cohérence,
- détruit la session si un port est violé,
- bloque l'IP hostile dans le pare-feu du système d'exploitation.

9.3.4. Ils sont non transmissibles

Un port directionnel :

- ne peut pas être envoyé,
- ne peut pas être copié,
- ne peut pas être exporté,
- ne peut pas être dérivé.

Il n'existe **que localement**, dans le tore.

9.4. Mutation des ports directionnels

Les ports directionnels mutent :

9.4.1. Lors du renouvellement du tore

Chaque fois que le tore privé est régénéré :

- un nouveau set de ports étendus est généré,

- tous les ports précédents deviennent invalides.

9.4.2. Lors d'une nouvelle connexion

À chaque nouvelle session :

- un nouveau port étendu est généré,
- une seule connexion est autorisée sur ce port,
- toute tentative de seconde connexion → destruction immédiate.

9.5. Ports directionnels et transmission

Host → Client

Le Host ouvre un port directionnel étendu. Le Client doit répondre **sur ce même port**.

Client → Host

Le Client renvoie la clé sur ce port. Le Host valide la cohérence.

Host → Client

Le Host renvoie la clé finale sur le même port.

Si un port directionnel est violé :

- la session est détruite,
- le Tore Privé est effacé,
- l'IP est bloquée.

9.6. Ports directionnels et multi-tore

Dans un environnement multi-tore :

- chaque branche possède ses propres ports directionnels,
- les ports ne sont pas compatibles entre branches,
- les ports ne peuvent pas être dérivés d'une branche à l'autre.

Un port directionnel est **strictement local** à sa branche.

9.7. Ports directionnels et sécurité post-quantique

Les ports directionnels sont inviolables par un ordinateur quantique :

- impossible de dériver un port étendu,

- impossible de prédire une mutation,
- impossible d'explorer le tore,
- impossible de reconstruire les ports internes.

9.8. Ports directionnels et sécurité photonique

Les ordinateurs photoniques peuvent casser :

- RSA,
- ECC,
- TLS,
- SSH.

Mais ils ne peuvent pas :

- analyser une géométrie directionnelle,
- dériver un tore,
- reconstruire un port directionnel,
- prédire un port étendu muté.

Les ports directionnels sont **structurellement inviolables**.

9.9. Résumé du Chapitre 9

Les ports directionnels :

- sont des ports classiques étendus à plusieurs milliards de possibilités,
- mutent à chaque renouvellement du tore,
- mutent à chaque nouvelle connexion,
- n'autorisent **qu'une seule connexion par port**,
- sont non scannables,
- non dérivables,
- non falsifiables,
- validés par la Forteresse de Verre,
- inviolables par conception.

Ils constituent la **couche de communication interne** de TorusNet, totalement indépendante des protocoles réseau classiques.

Chapitre 10 — Authentification DNS et HTTPS (Couche de Confiance Temporaire)

10.1. Vue d'ensemble

Avant que la géométrie directionnelle ne prenne le contrôle total de la session, une **couche de confiance temporaire** est nécessaire pour :

- identifier l'hôte demandé,
- éviter les redirections malveillantes,
- empêcher les attaques MITM classiques,
- sécuriser la transmission de la première clé directionnelle,
- garantir que le client contacte bien le bon serveur.

Cette couche repose sur deux technologies de l'Internet classique :

- **DNS** (résolution du nom de domaine),
- **HTTPS** (certificat TLS).

Ces technologies **ne font pas partie de la mécanique directionnelle**. Elles servent uniquement de **sas d'entrée**, avant que le protocole n'active :

- le Tore d'Authentification,
- le Tore Privé,
- la Forteresse de Verre,
- les IDs internes,
- l'ID global,
- l'ID unique du tore.

Une fois la première clé validée, **DNS et HTTPS disparaissent complètement**.

10.2. Rôle du DNS dans TorusNet

DNS est utilisé pour une seule raison :

Identifier l'hôte demandé avant d'entrer dans la géométrie directionnelle.

10.2.1. Vérification du domaine

Lorsqu'un client contacte un serveur :

1. Le DNS résout le nom de domaine.
2. Le protocole vérifie que ce domaine appartient à la liste des hôtes de confiance.
3. Si le domaine est inconnu ou suspect :
 - la connexion passe en mode confirmation utilisateur,
 - aucune clé directionnelle n'est transmise.

10.2.2. Protection contre les redirections

DNS empêche :

- les redirections malveillantes,
- les faux domaines,
- les hôtes clonés,
- les détournements de trafic.

10.2.3. Rôle strictement temporaire

Une fois la première clé validée :

DNS n'est plus utilisé. La géométrie directionnelle prend le relais.

10.3. Rôle du HTTPS dans TorusNet

HTTPS est utilisé pour :

Sécuriser la transmission de la première clé directionnelle.

10.3.1. Vérification du certificat

Le client vérifie :

- le certificat TLS,
- la validité du domaine,
- l'absence d'interception MITM classique.

Si le site n'a pas de HTTPS :

- la connexion passe en mode confirmation utilisateur,

- aucune clé directionnelle n'est envoyée.

10.3.2. Protection contre les attaques classiques

HTTPS protège contre :

- les interceptions,
- les injections,
- les manipulations de trafic,
- les attaques de downgrade.

10.3.3. Rôle strictement temporaire

Une fois la première clé transmise :

HTTPS disparaît. La session devient 100 % directionnelle.

10.4. Interaction DNS ↔ HTTPS ↔ Tore d'Authentification

La séquence complète est :

1. DNS identifie l'hôte.
2. HTTPS sécurise la première transmission.
3. Le Tore d'Authentification valide la première clé.
4. La Forteresse de Verre active la session directionnelle.
5. Le Tore Privé prend le contrôle.
6. DNS et HTTPS sont abandonnés.

10.4.1. Cas de confiance totale

- DNS valide
- HTTPS valide → La première clé est transmise automatiquement.

10.4.2. Cas de confiance partielle

- DNS valide
- HTTPS absent → Confirmation utilisateur obligatoire.

10.4.3. Cas de suspicion

- DNS suspect

- HTTPS absent ou invalide → Connexion bloquée jusqu'à validation explicite.

10.5. Pourquoi TorusNet utilise encore DNS et HTTPS

TorusNet est une architecture directionnelle post-Internet. Cependant, l'écosystème actuel impose :

- l'usage de noms de domaine,
- l'usage de certificats TLS,
- l'usage de serveurs HTTP(S).

DNS et HTTPS servent donc de **pont** entre :

- l'Internet classique,
- l'Internet directionnel.

10.5.1. Objectif final : disparition totale

Dans les versions futures :

- DNS sera remplacé par un **DNS directionnel**,
- HTTPS sera remplacé par une **validation directionnelle native**,
- les certificats seront remplacés par des **IDs directionnels**,
- les connexions seront entièrement **géométriques**.

10.6. Interaction avec les mises à jour directionnelles

MAJ 1 — Inversion et mutation d'ID

Une fois la première clé validée :

- l'ID de la case de départ mute,
- l'ID global est créé,
- DNS et HTTPS ne sont plus nécessaires.

MAJ 2 — Session directionnelle autonome

La session devient :

- continue,
- vivante,

- autonome.

Résumé du Chapitre 10

Le chapitre 10 établit que :

- DNS et HTTPS ne servent qu'à sécuriser la **première clé**,
- ils disparaissent immédiatement après validation,
- la session devient **100 % directionnelle**,
- la Forteresse de Verre prend le contrôle,
- le Tore Privé devient l'unique couche de communication,
- les IDs internes remplacent les certificats,
- les ports directionnels remplacent les sockets,
- la géométrie directionnelle remplace la cryptographie.

Chapitre 11 — Confiance à Long Terme et Reconnexion Directionnelle

11.1. Vue d'ensemble

La reconnexion directionnelle doit garantir que **le tore privé appartient réellement au client**, et qu'il n'a pas été :

- copié,
- volé,
- transféré,
- cloné,
- déplacé sur une autre machine.

Pour cela, TorusNet utilise une **triple identité matérielle directionnelle** :

1. ID carte mère
2. ID processeur
3. Adresse MAC

Cette triple identité rend le tore privé **intransférable** et **inutilisable ailleurs**.

11.2. La triple identité matérielle

Lors d'une reconnexion, le client envoie au Host :

1. **L'ID de la carte mère**
2. **L'ID du processeur**
3. **L'adresse MAC réelle**

Ces trois identifiants forment une **signature matérielle directionnelle**, impossible à falsifier.

11.3. Rôle de chaque identifiant

11.3.1. ID de la carte mère

- Gravé dans le firmware
- Non modifiable
- Non virtualisable

11.3.2. ID du processeur

- Identifiant unique du CPU
- Non clonable
- Non simulable

11.3.3. Adresse MAC

- Identité réseau matérielle
- Empêche l'utilisation du tore sur une autre interface

11.4. Pourquoi cette triple identité est parfaite

Même si un attaquant copie :

- le tore privé,
- les fichiers directionnels,
- les ports étendus,
- les IDs internes,

il ne pourra jamais reproduire :

- l’ID carte mère,
- l’ID processeur,
- l’adresse MAC,
- la cohérence fractale du tore.

11.5. Séquence de reconnexion

Lors d’une reconnexion, le client envoie :

- ID carte mère
- ID processeur
- Adresse MAC
- Clé réseau de reconnexion
- ID du tore privé
- ID global de session

Le Host compare les identifiants et valide ou détruit la session.

11.6. Mise à jour directionnelle des identifiants matériels

(Nouvelle section — V5.3)

Un utilisateur peut changer :

- sa carte mère,
- son processeur,
- sa carte réseau (donc sa MAC).

Pour éviter de perdre son tore privé, TorusNet permet une **mise à jour directionnelle contrôlée** des identifiants matériels.

11.6.1. Règle fondamentale :

Pour mettre à jour un identifiant, il faut prouver son identité avec les deux autres.**

Cela donne trois cas :

Cas 1 — Mise à jour de l’ID du processeur

Pour mettre à jour l'ID CPU, le client doit fournir :

- **l'ID de la carte mère,**
- **l'adresse MAC,**
- **l'ancien ID CPU,**
- **le nouveau ID CPU.**

Le Host vérifie :

- cohérence fractale,
- cohérence directionnelle,
- cohérence matérielle.

Si tout est cohérent :

→ **mise à jour autorisée** → le tore privé est réindexé → les ports directionnels sont régénérés
→ l'ID CPU est remplacé

Cas 2 — Mise à jour de l'ID de la carte mère

Pour mettre à jour l'ID carte mère, le client doit fournir :

- **l'ID du processeur,**
- **l'adresse MAC,**
- **l'ancien ID carte mère,**
- **le nouveau ID carte mère.**

Même logique : → validation directionnelle → mise à jour autorisée → tore privé réindexé

Cas 3 — Mise à jour de l'adresse MAC

Pour mettre à jour la MAC, le client doit fournir :

- **l'ID carte mère,**
- **l'ID processeur,**
- **l'ancienne MAC,**
- **la nouvelle MAC.**

Même logique : → validation directionnelle → mise à jour autorisée → tore privé réindexé

11.6.2. Pourquoi cette règle est inviolable

Un attaquant ne peut pas :

- simuler deux identifiants matériels cohérents,
- falsifier deux identifiants matériels simultanément,
- reproduire la cohérence fractale du tore,
- passer la validation de la Forteresse de Verre.

Même avec un tore privé volé, il ne pourra jamais fournir :

- deux identifiants matériels valides,
- cohérents entre eux,
- cohérents avec le tore privé.

11.7. Interaction avec la Forteresse de Verre

La Forteresse de Verre :

- surveille les identifiants matériels,
- valide les mises à jour,
- détecte les falsifications,
- détruit le tore privé si incohérence,
- bloque l'IP hostile.

11.8. Résumé du Chapitre 11

La reconnexion directionnelle repose sur :

- une triple identité matérielle,
- une validation fractale,
- une cohérence directionnelle,
- une impossibilité de transfert du tore privé.

La mise à jour des identifiants matériels est possible **uniquement** si :

- deux identifiants matériels sont encore valides,
- la cohérence fractale est intacte,

- la Forteresse de Verre valide la requête.

Cette règle rend TorusNet :

- inviolable,
- intransférable,
- résistant au vol,
- résistant au clonage,
- résistant à la virtualisation.

Update 6 — Version 6 du protocole TorusNet

Refonte du seed directionnel et introduction du moteur d'exposants multi-étages

Cette mise à jour définit la structure complète du **seed directionnel de 70 bits**, utilisé pour générer les trajectoires internes du tore privé et du tore public. Le moteur d'exposants introduit dans cette version permet de représenter des valeurs hyper-exponentielles impossibles à inverser ou dériver par toute technologie connue (classique, quantique ou photonique).

1. Structure de la clé directionnelle (128 bits)

La clé directionnelle TorusNet est composée de :

- **2 bits — Type de clé**
- **20 bits — Point de départ directionnel**
- **20 bits — Point d'arrivée directionnel**
- **16 bits — ID de case visée**
- **70 bits — Seed directionnel**

Total :

128 bits

2. Structure du seed directionnel (70 bits)

Le seed directionnel contient cinq composantes :

1. Base exponentielle (4 bits)

2. Sélection de la tour fixe (4 bits)
3. Hauteur de la tour fixe (2 bits)
4. Étages dynamiques (52 bits)
5. Modulation finale (addition + multiplication), encodée dans le seed (4 bits + 4 bits)

Ces éléments forment ensemble le seed directionnel complet de 70 bits.

2.1 Base exponentielle — 4 bits

Les 4 bits définissent une base parmi 16 valeurs hiérarchiques :

1,2,3,4,5,6,7,8,9,10,100,1000,10 000,100 000,1 000 000,10 000 000

2.2 Sélection de la tour fixe — 4 bits

Les 4 bits permettent :

- d'activer une tour fixe (8 valeurs possibles),
- ou de ne pas en utiliser (8 valeurs possibles).

Tours fixes disponibles :

$1^{2^{3^4}}, 2^{3^{4^5}}, 3^{4^{5^6}}, 4^{5^{6^7}}, 5^{6^{7^8}}, 6^{7^{8^9}}, 7^{8^{9^{10}}}, 8^{9^{10^{11}}}$

2.3 Hauteur de la tour fixe — 2 bits

Détermine le nombre d'étages utilisés :

- 00 → 1 étage
- 01 → 2 étages
- 10 → 3 étages
- 11 → 4 étages

2.4 Étages dynamiques — 52 bits

Les **52 bits** sont divisés en **6 étages dynamiques**, chacun représenté par une **valeur brute sur 8 bits** :

$D_1, D_2, D_3, D_4, D_5, D_6 \in \{0, 1, \dots, 255\}$

Règle directionnelle :

- si un étage dynamique vaut **0**, il est **ignoré**,

- si sa valeur est > 0 , il est **actif** dans la tour exponentielle.

Ces 6 étages constituent la partie dynamique du moteur d'exposants.

2.5 Modulation finale (encodée dans le seed)

Deux valeurs supplémentaires sont encodées dans le seed :

- **Addition (A)** — 4 bits $\rightarrow$ valeur de 0 à 15
- **Multiplication (M)** — 4 bits $\rightarrow$ valeur de 0 à 15

Ces 8 bits font partie des 70 bits du seed et sont appliqués **après** la construction complète de la tour exponentielle.

La valeur exponentielle finale devient :

$$E_{\text{final}} = (X + A) \times M$$

où **X** représente la tour complète :

$$X = \text{Base}^{\text{Tour fixe}}^{D_1^{D_2^{D_3^{D_4^{D_5^{D_6}}}}}}$$

Règles :

- si **A = 0**, l'addition n'a aucun effet,
- si **M = 0**, l'étage final est **ignoré**,
- si **M > 0**, l'étage final est **actif**.

3. Structure exponentielle finale

Avec tour fixe :

$$\text{Base}^{e_1^{e_2^{e_3^{e_4^{D_1^{D_2^{D_3^{D_4^{D_5^{D_6}}}}}}}}}}$$

Sans tour fixe :

$$\text{Base}^{D_1^{D_2^{D_3^{D_4^{D_5^{D_6}}}}}}$$

Puis modulation :

$$E_{\text{final}} = (X + A) \times M$$

4. Capacité de représentation

- **Espace brut :**

$$2^{52}$$

- **Espace exponentiel :** dépasse largement les capacités cryptographiques, quantiques et photoniques.

5. Impact sur le tore privé

Le tore privé utilise :

- la base exponentielle,
- la tour fixe (si activée),
- les 6 étages dynamiques,
- la modulation finale (A et M),
- la règle d'ignorance des étages nuls,

pour déterminer la trajectoire directionnelle interne.

Chaque seed produit une **direction unique**, impossible à prédire ou reproduire sans la structure complète.

6. Conclusion de l'Update 6

Update 6 introduit :

- un seed directionnel de **70 bits**,
- une base exponentielle hiérarchique,
- une tour fixe optionnelle,
- une hauteur configurable,
- **6 étages dynamiques sur 8 bits**,
- une modulation finale encodée dans le seed (**4 bits d'addition + 4 bits de multiplication**),
- une suppression automatique des étages nuls.

Cette structure renforce la nature **inviolable**, **non dérivable**, et **géométriquement imprévisible** du protocole TorusNet.